

Agentic AI: Foundations, Patterns & Architecture

A practitioner's reference for terminology, design patterns, multi-agent architecture, and the current state of the art

Prepared by Ajith Kattil | Neural Labs

September 2026

Contents

1. What "Agentic AI" Actually Means

Most confusion in this space comes from one thing: "agentic" gets used for four different systems that behave very differently. Before any pattern or diagram, it helps to pin down where a given system sits on a single spectrum: how much control over its own next action the system has.

1.1 The autonomy spectrum

- Scripted workflow - fixed sequence of steps, no LLM decides what happens next (e.g. a Zapier chain, a CI/CD pipeline).
- LLM-augmented workflow - one or more LLM calls embedded in an otherwise fixed pipeline (e.g. "summarize, then classify, then email"). The model reasons within a step but does not choose which step runs.
- Tool-calling assistant - a single LLM decides which tool to call from a fixed set, usually one call at a time, often with a human confirming each action.
- Autonomous agent - the LLM runs a loop: it decides the next action, executes it, observes the result, and decides again, continuing until it judges the goal met or a stop condition fires. This is what the rest of this document calls an agent.
- Multi-agent system - two or more agents (each running their own loop) coordinate, via a fixed topology, to solve a task no single agent handles well alone.

Working analogy: a scripted workflow is a train on fixed tracks - it always arrives the same way. An agent is a car with a driver: given a destination and a map (tools + memory), it chooses the route turn by turn based on what it observes. A multi-agent system is a small team of drivers, each with a role, coordinating to get a convoy there.

1.2 A working definition

An agent = an LLM given (a) a goal, (b) access to tools/actions, (c) some form of memory, and (d) the ability to decide its own next action in a loop - rather than following a path a human hard-coded in advance.

Everything below - ReAct, Plan-and-Execute, Reflection, multi-agent topologies - is a different answer to the same three engineering questions: how does the agent decide what to do next, how much of the plan is fixed in advance, and who (or what) checks its work before it acts.

2. Core Building Blocks

Every agentic system, regardless of framework, is assembled from the same six components. Naming them consistently is most of what resolves the terminology confusion between vendor docs.

Component	Role	Typical implementation
Model	The reasoning/decision engine - turns context into the next thought or action.	Claude, GPT, Gemini, open-weight models
Tools / Actions	Functions the agent can invoke to affect or read the outside world.	Function calling, MCP servers, plain REST/SDK calls
Memory	State carried across steps or sessions.	Context window, vector DB, key-value store, checkpoint store
Orchestrator / Control flow	Decides what runs next: the agent itself, a graph, or a supervising process.	LangGraph, a state machine, a simple while-loop
Guardrails	Validates inputs/outputs against policy before they reach a tool or the user.	Schema validation, PII filters, content classifiers, confidence gates

Observability	Tracing, logging, and evaluation of agent behavior in production.	LangSmith, Langfuse, Arize, OpenTelemetry traces
---------------	---	--

3. Foundational Single-Agent Patterns

These four patterns cover the vast majority of what a single agent does today. Multi-agent systems (Section 4) are typically built by wiring several of these loops together.

3.1 The Core Agent Loop

Every pattern below is a variation on one loop: perceive, reason, act, observe, repeat. What differs is how much structure gets imposed on the "reason" step and how the loop decides to stop.

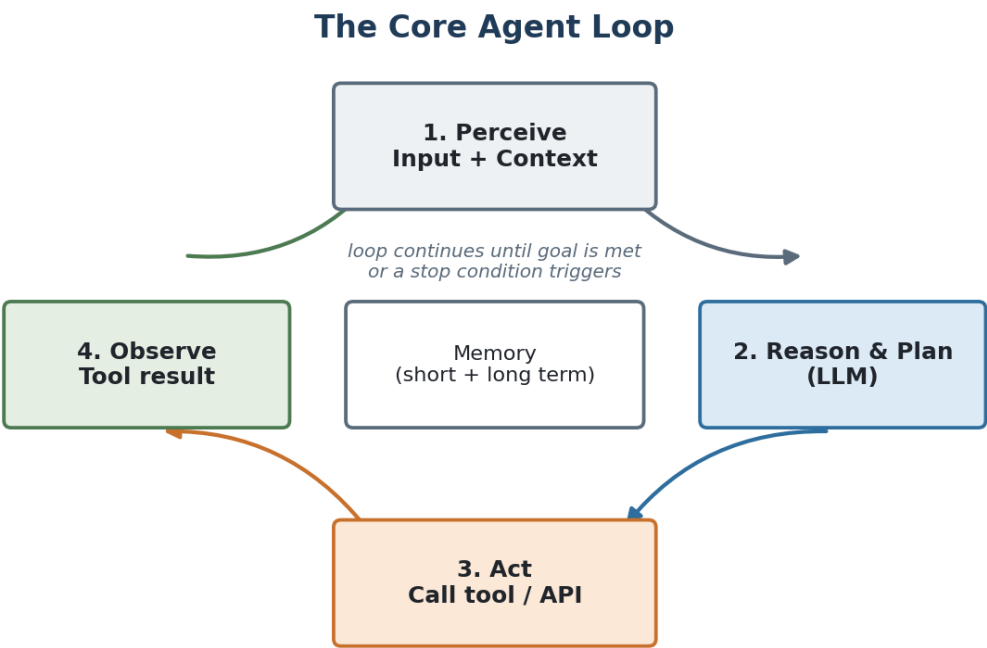
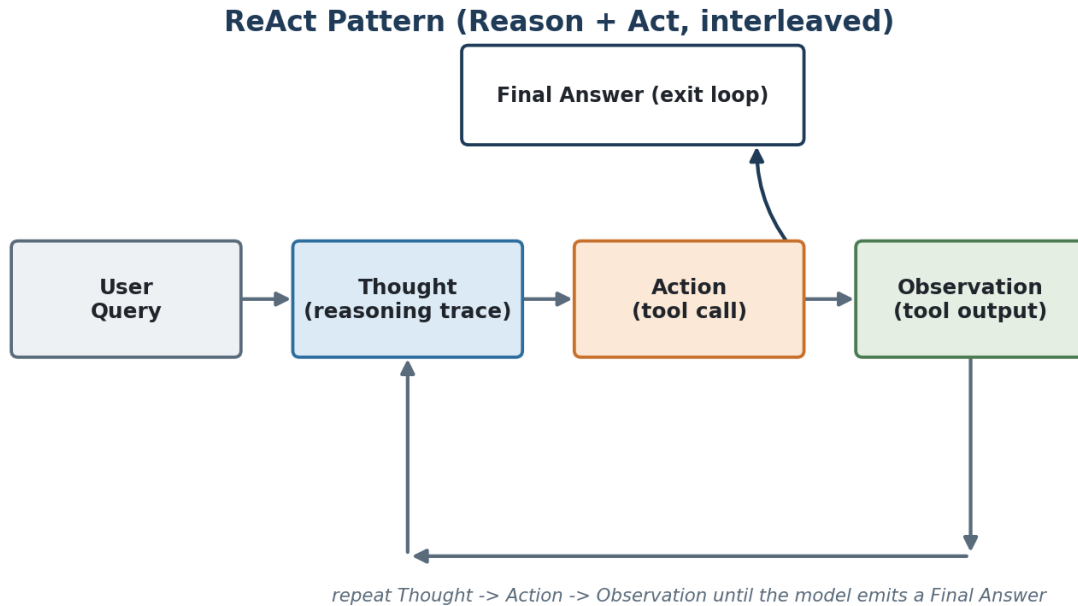


Figure 1. The core agent loop underlying every pattern in this document.

3.2 ReAct (Reason + Act)

ReAct interleaves a visible reasoning step ("Thought") with a tool call ("Action") and its result ("Observation"), repeating until the model decides it has enough to answer. It is the simplest pattern to implement and debug because every step is visible and the loop has no separate planning phase - the agent re-plans, implicitly, at every turn.



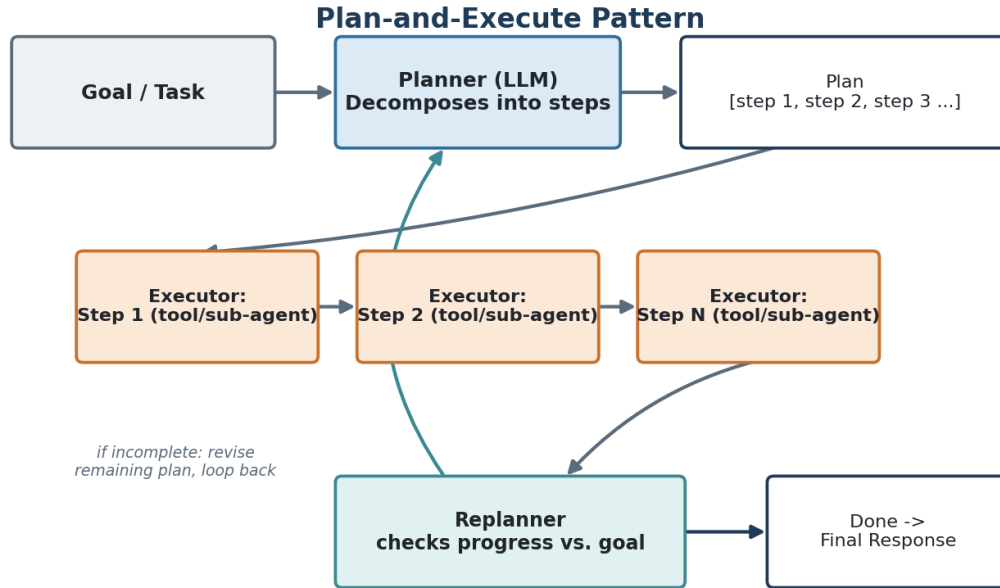
Popularized by Yao et al. 2022; still the default single-agent tool-use loop in LangGraph, OpenAI Agents SDK, Bedrock Agents.

Figure 2. ReAct - reasoning and acting interleaved, one step at a time.

- Best for: short-to-medium tasks where the next best action genuinely depends on what the last tool call returned (search-then-refine, debugging, customer support lookups).
- Weakness: no global plan, so it can wander or loop on long-horizon tasks; every step costs a full model call.

3.3 Plan-and-Execute

A planner produces the full list of steps up front; a (often cheaper/smaller) executor works through them one at a time; a replanner checks progress and revises the remaining plan if something failed or the situation changed. This trades some adaptability for lower cost and much better controllability on long-horizon tasks.



Separates 'what to do' (planning) from 'how to do each step' (execution) - cheaper and more controllable than pure ReAct for long-horizon tasks.

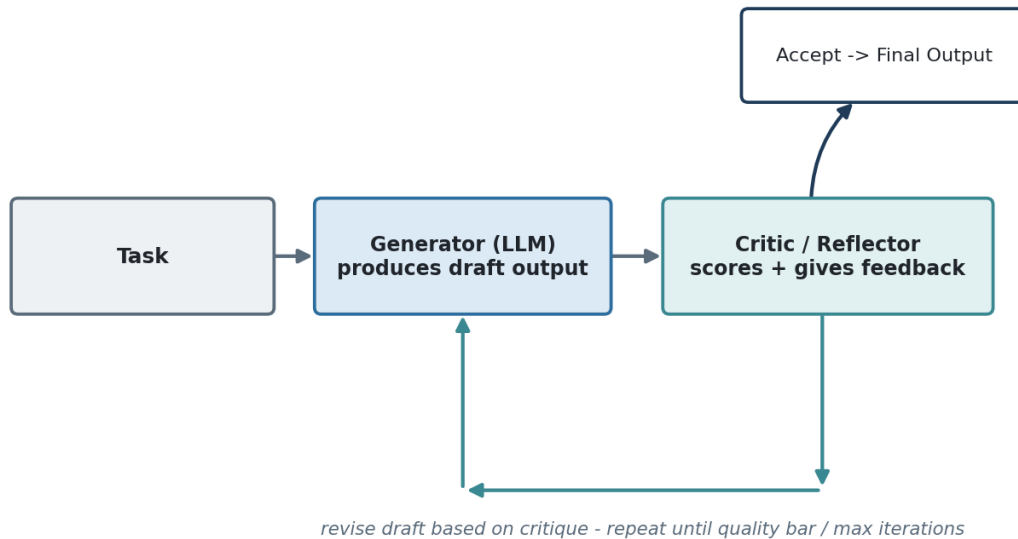
Figure 3. Plan-and-Execute - separates deciding what to do from doing it.

- Best for: multi-step tasks with a knowable shape in advance (research reports, multi-file code changes, claim-processing workflows).
- Weakness: a bad initial plan can waste a lot of execution before the replanner catches it; needs a real replanning step, not just retries.

3.4 Reflection / Self-Critique

A generator produces a draft; a critic (a second LLM call, an LLM-as-judge, or a rule-based verifier) scores it and returns specific feedback; the generator revises. This repeats until the critic is satisfied or a max-iteration limit is hit.

Reflection / Self-Critique Pattern



Used in Reflexion, Self-Refine, CRITIC; pairs well with an LLM-as-judge or a rule-based verifier as the critic.

Figure 4. Reflection - a critic loop that catches errors before they reach the user.

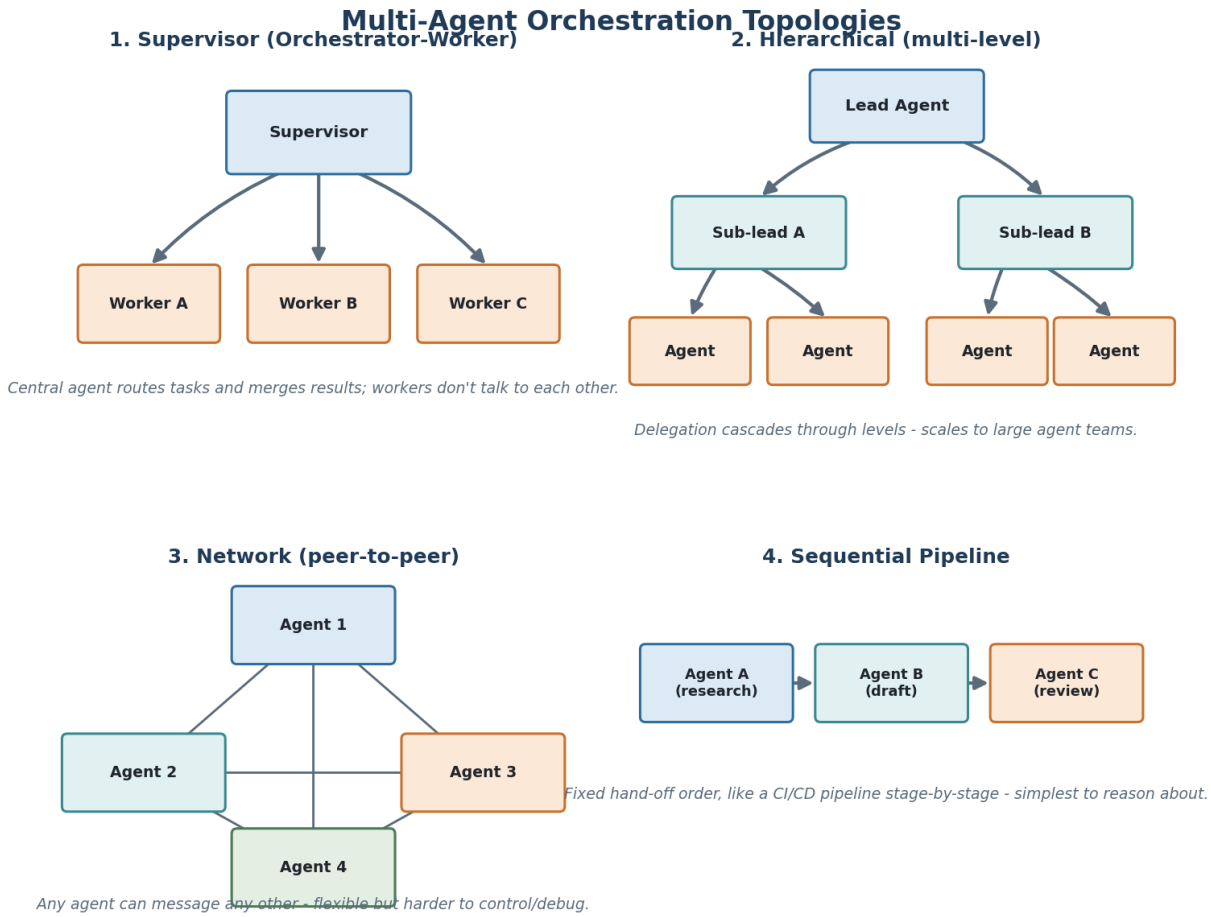
- Best for: quality-sensitive single-shot outputs - code review, summarization accuracy, compliance-sensitive drafting.
- Weakness: doubles-or-triples cost per output; a weak critic gives false confidence.

3.5 Tool Use / Function Calling (the substrate underneath all of the above)

Function calling is not a separate "pattern" so much as the mechanism the other four patterns run on: the model is given a set of tool schemas (name, description, JSON parameters) and, instead of only producing text, can emit a structured request to invoke one. The orchestrator executes the real function, and the result is fed back into context as an observation. Modern models support parallel tool calls (several tools requested in one turn) and forced/structured output modes that constrain the response to a valid schema - both reduce the retry loops that made early tool-calling agents unreliable.

4. Multi-Agent Architectures

A single agent looping on one context window starts to strain once a task needs genuinely different expertise, needs to run sub-tasks in parallel, or would otherwise blow the context window with irrelevant history. Multi-agent systems solve this by giving each agent a narrower role and a fixed communication topology.



"Memory" is one of the most overloaded words in this space. It is useful to separate it the way operating systems separate RAM from disk: memory scoped to right now, memory scoped to this session, and memory that survives across sessions - further split by what kind of thing is being remembered.

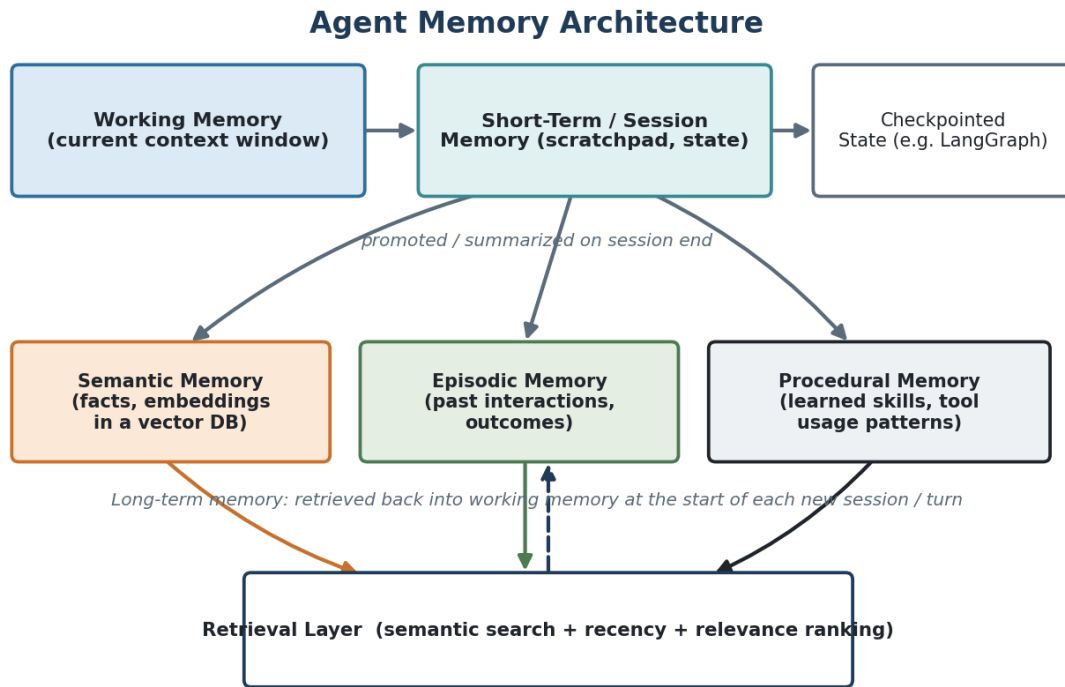


Figure 6. Agent memory, layered from the context window down to long-term stores.

Memory type	What it holds
Working memory	The literal current context window - everything the model sees on this call.
Short-term / session memory	A scratchpad or state object carried across turns in one session (LangGraph state, a conversation buffer).
Semantic memory	Facts and general knowledge, usually as embeddings in a vector database, retrieved by similarity.
Episodic memory	Records of specific past interactions and their outcomes - what was tried, what worked.
Procedural memory	Learned patterns of tool use / skills - closer to a cached strategy than a fact.

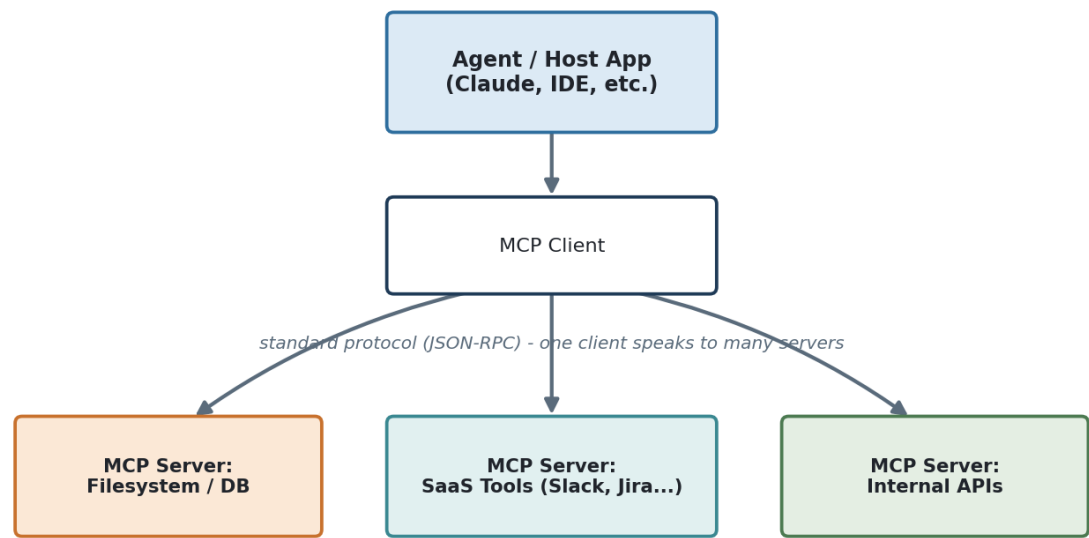
6. Standardizing Tools and Agent-to-Agent Communication

Before 2024, every framework wired an agent to its tools with bespoke glue code. Two protocols now standardize the two different connection points this creates, and it is worth being precise about which is which.

6.1 MCP - Model Context Protocol (agent to tool/data)

MCP, introduced by Anthropic in late 2024, standardizes how an agent (the "host", via an "MCP client") talks to external tools and data sources ("MCP servers") over a common JSON-RPC-based schema. A server exposes three kinds of primitives - Tools (callable functions), Resources (readable context/data), and Prompts (reusable prompt templates) - and any MCP-compatible host can use any MCP-compatible server without custom integration code.

Standardized Tool/Context Access (MCP pattern)



Each server exposes Tools, Resources, and Prompts through one schema - agent code stays the same as servers change.

Figure 7. MCP standardizes the agent-to-tool boundary.

6.2 A2A - Agent2Agent protocol (agent to agent)

A2A, introduced by Google in 2025 and since contributed to the Linux Foundation, standardizes the other boundary: how independently-built agents (potentially from different vendors or frameworks) discover each other's capabilities and exchange tasks and results, without either side seeing the other's internal reasoning or tools. Where MCP is about giving one agent more tools, A2A is about letting separate agents collaborate as peers - the network topology in Section 4, but across organizational or vendor boundaries instead of inside one codebase.

	MCP	A2A
Connects	Agent <-> tools/data	Agent <-> agent
Unit of exchange	Tool calls, resources, prompts	Tasks, artifacts, status updates
Typical use	Giving Claude/an IDE access to a database, filesystem, SaaS API	A travel-booking agent delegating to a payment agent built by a different team

7. Agentic RAG

Classic RAG is a fixed pipeline: embed the query, retrieve top-k chunks, stuff them into the prompt, generate. Agentic RAG replaces the fixed pipeline with a loop the agent controls: it can choose which retrieval method(s) to use, judge whether what it retrieved is good enough, and decide to retrieve again, reformulate the query, or escalate - before it ever generates a final answer.

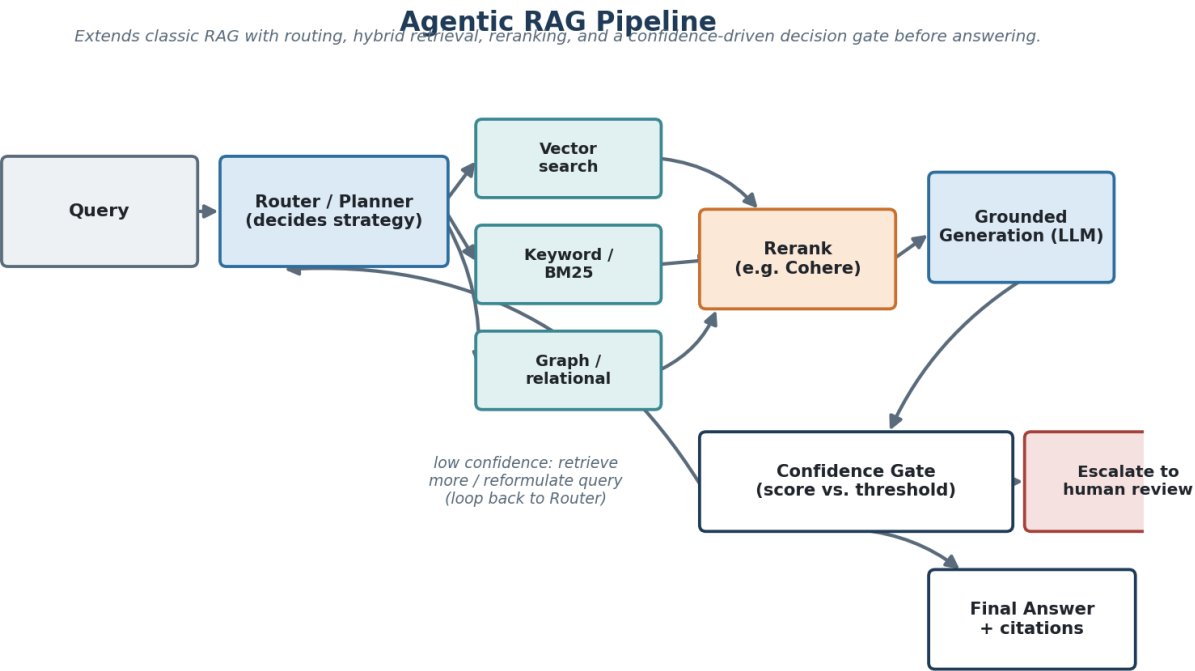


Figure 8. Agentic RAG - routing, hybrid retrieval, reranking, and a confidence gate around generation.

- Router / Planner - decides retrieval strategy per query rather than always doing the same lookup.
- Hybrid retrieval - combines vector (semantic) search with keyword/BM25 and, increasingly, graph or relational lookups, since no single retrieval method is best for every query shape.
- Reranking - a cross-encoder (e.g. Cohere Rerank) reorders retrieved candidates for relevance before they reach the generator, which consistently improves grounding over vector-similarity ordering alone.
- Confidence gate - a score against the retrieved evidence (or the generated answer) decided by threshold: high enough, answer; too low, retrieve more, reformulate, or escalate to a human.

This is the same shape used in hybrid-retrieval, confidence-gated designs for domains like claims authentication or fraud review, where a wrong answer is expensive enough that "retrieve again or ask a human" has to be a first-class path, not an afterthought.

8. Orchestration Framework Landscape

Frameworks differ mainly in how explicit the control flow is. This is the axis that matters most when picking one - not feature checklists.

Framework	Control model	Best for	Notable feature
LangGraph	Explicit state graph (nodes + edges), you define control flow	Complex, long-running, or human-in-the-loop workflows needing precise control	Native checkpointing + interrupt_before for pausing on human review
CrewAI	Role-based agents with tasks assigned by a crew/process	Quickly standing up a supervisor/worker or pipeline team	Low-code role/task abstraction, fast to prototype

AutoGen / AG2	Conversable agents that message each other	Research-style multi-agent conversation, network topology	Strong support for the network/peer-to-peer pattern
OpenAI Agents SDK	Lightweight agent loop + handoffs between agents	Teams standardized on OpenAI models wanting minimal framework overhead	Built-in tracing and guardrails primitives
Google ADK	Hierarchical agent composition	Teams on Vertex AI / Gemini wanting native A2A support	First-class Agent2Agent protocol integration
Semantic Kernel	.NET/Python "plugin" + planner model	Enterprise .NET/Azure shops	Deep Azure/enterprise integration, plugin ecosystem
AWS Bedrock Agents	Managed agent runtime on Bedrock	AWS-native teams wanting a managed control plane	Managed action groups + knowledge base retrieval
Claude Agent SDK	Same harness Claude Code runs on, exposed for custom agents	Coding and computer-use agents needing file/bash/browser tools out of the box	Production hardening of the agent loop used by Claude Code itself

9. Guardrails, Evaluation & Production Concerns

The gap between a working demo and a production agent is almost entirely in this section - not in the pattern chosen.

9.1 Guardrails

- Input validation - schema and policy checks before a request reaches the model or a tool.
- Output validation - structured-output/schema enforcement so a tool call can't be malformed; content and PII filters before a response reaches the user.
- Confidence gates - a numeric threshold on retrieval or answer confidence that routes low-confidence cases to a fallback instead of answering wrong (Section 7).
- Circuit breakers - stop calling a failing downstream tool/service after N consecutive failures, fail fast, and surface the failure instead of retrying indefinitely.
- Human-in-the-loop interrupts - a designed pause point (e.g. LangGraph's `interrupt_before`) where the graph stops and waits for a human decision before a high-stakes action executes.

9.2 Evaluation

- LLM-as-judge - a separate model call scores outputs against a rubric; useful for scale, but the judge itself needs periodic calibration against human review.
- Golden datasets / regression suites - a fixed set of inputs with known-good outputs, run on every change, the same way unit tests gate a code change.
- Trace-based observability - full step-by-step traces of every agent run (LangSmith, Langfuse, Arize, or plain OpenTelemetry spans) - without this, debugging a multi-step agent in production is close to impossible.

9.3 Cost and latency control

- Model routing - send simple sub-tasks to a small/cheap model and only escalate to a frontier model when the task genuinely needs it.
- Caching - cache retrieval results and, where safe, full responses for repeated or near-duplicate queries.

- Batching / parallel tool calls - run independent tool calls concurrently instead of serially inside one turn.

10. What's New: State of the Art Through Late 2026

A fast-moving snapshot - the patterns in Sections 3-7 are stable; this section is the part worth re-checking every few months.

- MCP ecosystem maturity - remote/hosted MCP servers, an official registry, and OAuth-based auth have moved MCP from "Anthropic's protocol" to a de facto cross-vendor standard adopted by OpenAI, Google, and Microsoft tooling.
- A2A protocol - now under the Linux Foundation, giving agents built on different stacks a standard way to discover and delegate to each other (Section 6.2).
- Computer-use / browser agents - agents that operate a GUI directly (mouse/keyboard/screenshots) rather than only calling APIs, closing the gap for the many enterprise systems with no API at all.
- Structured outputs and constrained decoding - grammar- or schema-constrained generation makes tool-call arguments reliably valid JSON, removing a large fraction of the retry loops that plagued 2023-era agents.
- Long-horizon agents with hierarchical memory - checkpointed state plus tiered memory (Section 5) lets agents run for hours across many context-window "generations" instead of being capped by a single window.
- Evaluation-driven development - agent evals run as CI gates before deployment, the same way unit tests gate code, rather than being a one-time pre-launch check.
- Cost-aware routing as a default - most production stacks now route by task difficulty rather than sending every step to the largest available model.

11. Terminology Quick-Reference

A single place to check a term's plain-English meaning when a vendor's docs use it slightly differently than another's.

Term	Plain-English definition
Agent	An LLM that decides its own next action in a loop, given a goal, tools, and memory.
Agentic workflow	A pipeline where an agent - not just a fixed script - controls at least one branching decision.
Tool / function calling	The mechanism by which a model requests a structured call to an external function.
Orchestrator	The code (graph, loop, or supervising agent) that decides what runs next.
ReAct	Reason + Act - interleaved thought/action/observation, one step at a time (Section 3.2).
Plan-and-Execute	Plan the full sequence up front, then execute and replan as needed (Section 3.3).
Reflection / self-critique	A generator-critic loop that revises output before it's finalized (Section 3.4).
Supervisor pattern	One agent routes tasks to and merges results from worker agents (Section 4).
Hierarchical multi-agent	Delegation cascades through multiple levels of lead/sub-lead agents (Section 4).

Human-in-the-loop	A designed pause point where a human approves or edits before the agent proceeds.
MCP	Model Context Protocol - standardizes agent-to-tool/data connections (Section 6.1).
A2A	Agent2Agent protocol - standardizes agent-to-agent task delegation across vendors (Section 6.2).
RAG	Retrieval-Augmented Generation - retrieve relevant context, then generate grounded in it.
Agentic RAG	RAG where the agent controls retrieval strategy, retries, and when to stop (Section 7).
Hybrid retrieval	Combining vector, keyword, and/or graph retrieval rather than relying on one method.
Reranking	Reordering retrieved candidates by a relevance model before generation.
Grounding	Constraining a generated answer to be supported by retrieved/cited evidence.
Confidence gate	A threshold check that routes low-confidence results to a fallback instead of answering.
Circuit breaker	Stops calling a repeatedly-failing dependency instead of retrying indefinitely.
Checkpointing	Persisting agent state so a long-running or paused task can resume exactly where it left off.
Context window	The literal text/tokens visible to the model on a given call ("working memory").
Semantic / episodic / procedural memory	Facts / past interactions / learned skills - the three long-term memory types (Section 5).
Guardrails	Validation and policy checks applied to inputs and outputs around the model.
LLM-as-judge	Using a separate model call to score another output against a rubric.

12. Where to Go Deeper

A short, high-signal reading list rather than an exhaustive bibliography:

- Anthropic, "Building Effective Agents" - the clearest practitioner treatment of when to use a workflow vs. an agent, and why simpler is usually better.
- The ReAct paper (Yao et al., 2022) - the original Reason+Act formulation still underlying most single-agent tool use.
- LangGraph documentation - the most complete reference implementation of explicit state-graph orchestration, checkpointing, and human-in-the-loop interrupts.
- modelcontextprotocol.io - the MCP specification and server/client reference implementations.
- Google's Agent2Agent (A2A) protocol documentation - the current cross-vendor agent-to-agent standard.

This document is intended as a living reference - the patterns in Sections 1-9 should stay stable for years; Section 10 is worth revisiting quarterly.